



Embedded Systems

Restaurant Delivery Robot



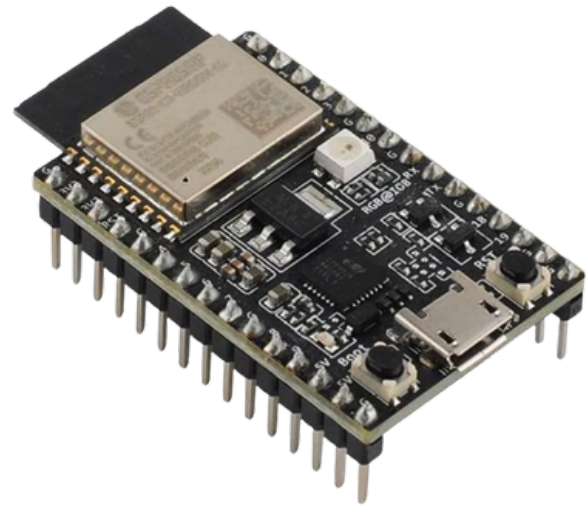
Group 3:
Bùi Trịnh Thế Viên
Trần Đăng Bách
Nguyễn Văn Quý
Nguyễn Trọng Nhân



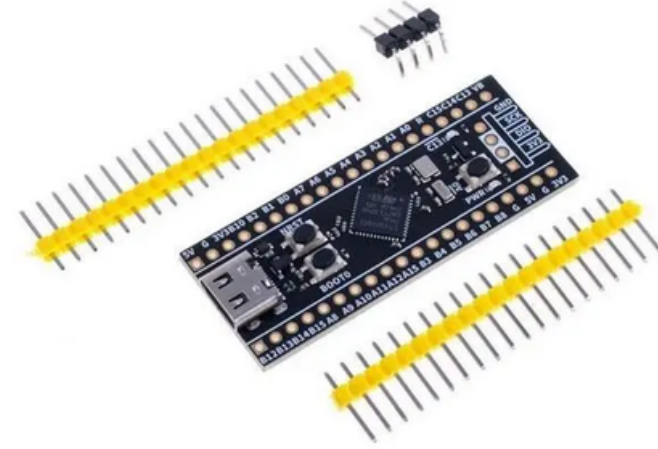
Table of Contents:

1. Introduction (overview + objectives)
2. System Architecture & Flowchart Diagrams
3. Use Case Diagram
4. Timeline
5. Project Task Allocation (tasks + contribution percentage)

COMPONENTS



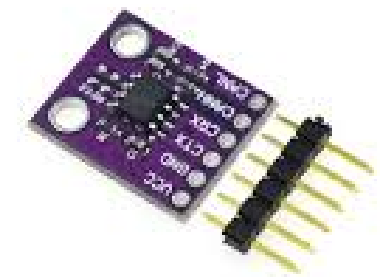
ESP32-C3



STM32H5



STLINK-V2



MCP2551



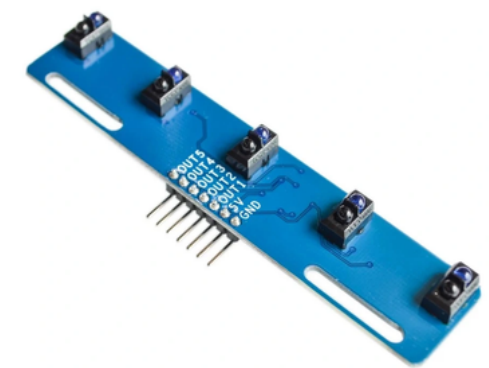
Resistor-120ohm



Driver motor



Motor N20



Line sensor

COMPONENTS



MPU6050



HC-SR04



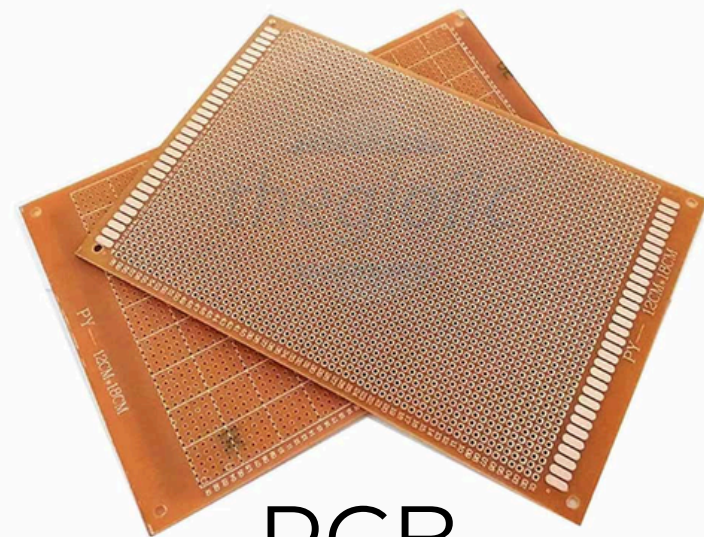
BUTTON



Pin Li-ion



LM2596



PCB



Wheel



Oled(I2C)

SOFTWARE & DEBUGGING TOOLS

Development IDEs

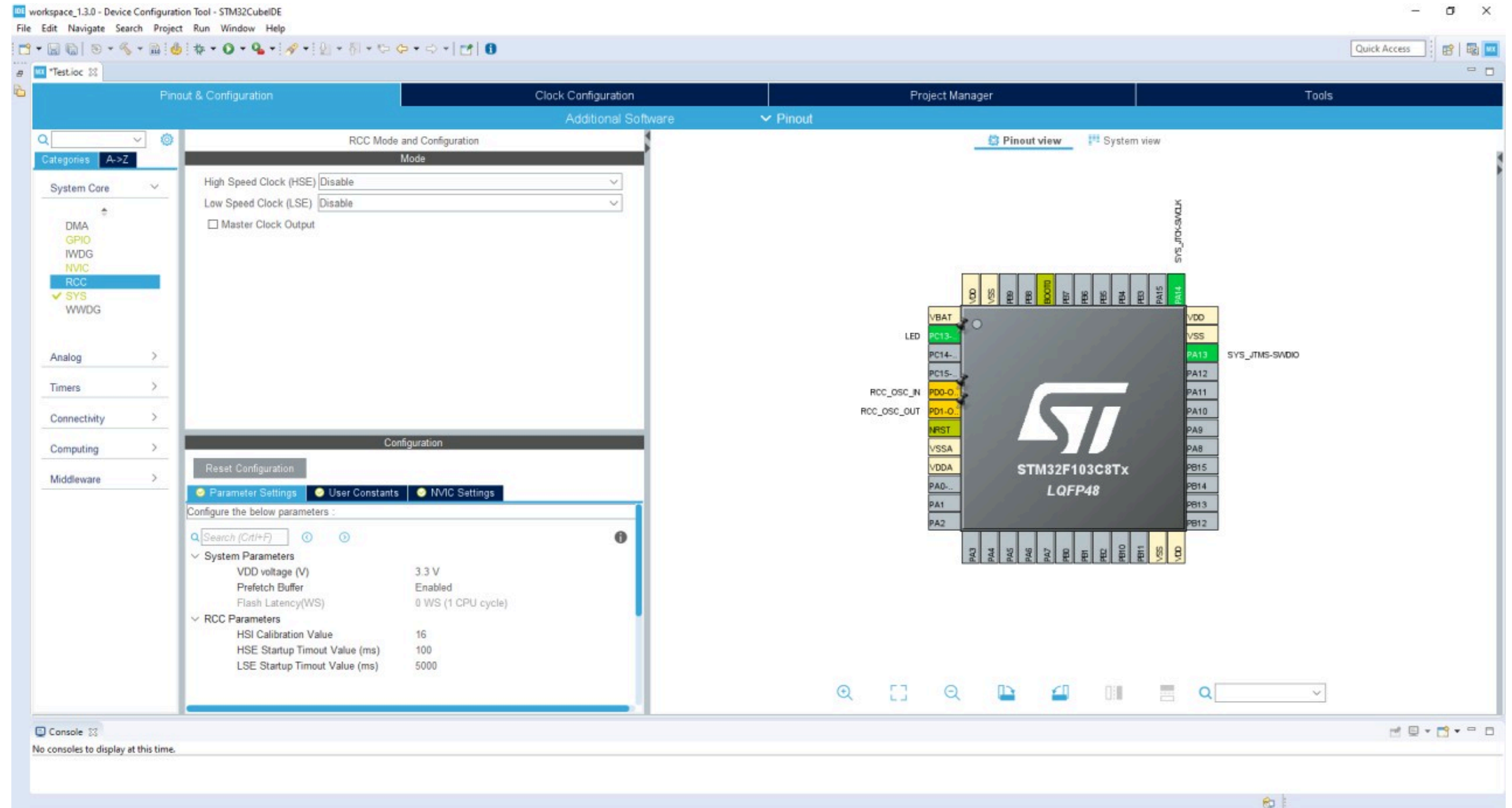
- STM32CubeIDE
- ESP-IDF
- VS Code + Espressif Extension

Firmware Libraries

- FreeRTOS
- STM32 HAL
- ESP-IDF TWAI driver
- ESP-IDF WiFi + lwIP
- MPU-6050/9250 driver
- SSD1306 driver
- Madgwick / Complementary Filter

Debugging & Analysis Tools

- OpenOCD + GDB
- Saleae Logic Analyzer
- ESP-IDF Monitor
- SEGGER SystemView



1. Introduction

Problem Statement:

In today's context, the restaurant service industry is gradually shifting toward automation in order to:

- Reduce labor costs
- Increase service efficiency
- Enhance customer experience

However, existing restaurant robot systems still have several limitations:

- Centralized processing → prone to overload, difficult to scale
- Slow response in emergency situations
- Lack of stability when operating in real-world environments

Therefore, there is a need to build a robot system that:

- Is capable of distributed processing
- Guarantees real-time performance
- Operates safely and reliably



1. Introduction

This project focuses on designing an automated delivery robot using a Multi-MCU architecture combined with CAN Bus communication.

The system is divided into 3 main blocks:

- MCU1 (ESP32): handles system logic, receives orders via WiFi, and coordinates the overall system
- MCU2 (STM32H5): controls movement, positioning, and navigation
- MCU3 (STM32H5): handles sensor processing and ensures safety

Each microcontroller operates independently and communicates with one another via CAN Bus, which helps to:

- Increase system stability
- Reduce processing load on each block
- Allow easy system expansion

The system uses FreeRTOS to manage multitasking and ensure real-time operation.



1. Introduction

Objectives:

- General Objective:
 - To build a restaurant delivery robot system capable of operating autonomously, stably, and safely in a real restaurant environment.
- Specific Objectives:
 - Receive and process orders via WiFi connection
 - Navigate the robot to the desired destination
 - Detect and avoid obstacles in real time
 - Perform an emergency stop when a hazard occurs
 - Return to the base position after completing a delivery
 - Enable communication between MCUs via CAN Bus
 - Apply FreeRTOS for efficient multitasking management



2.1. System Architecture

System Architecture: 3 nodes on CAN Bus:

- ESP32 (MCU1) — Brain / WiFi / Mission Controller / OLED Display
- STM32H5 (MCU2) — Motion & Navigation Controller
- STM32H5 (MCU3) — Perception & Safety Controller

Node	ID
ESP32	0x01
STM32H5	0x02
STM32H5	0x03

2.1. System Architecture

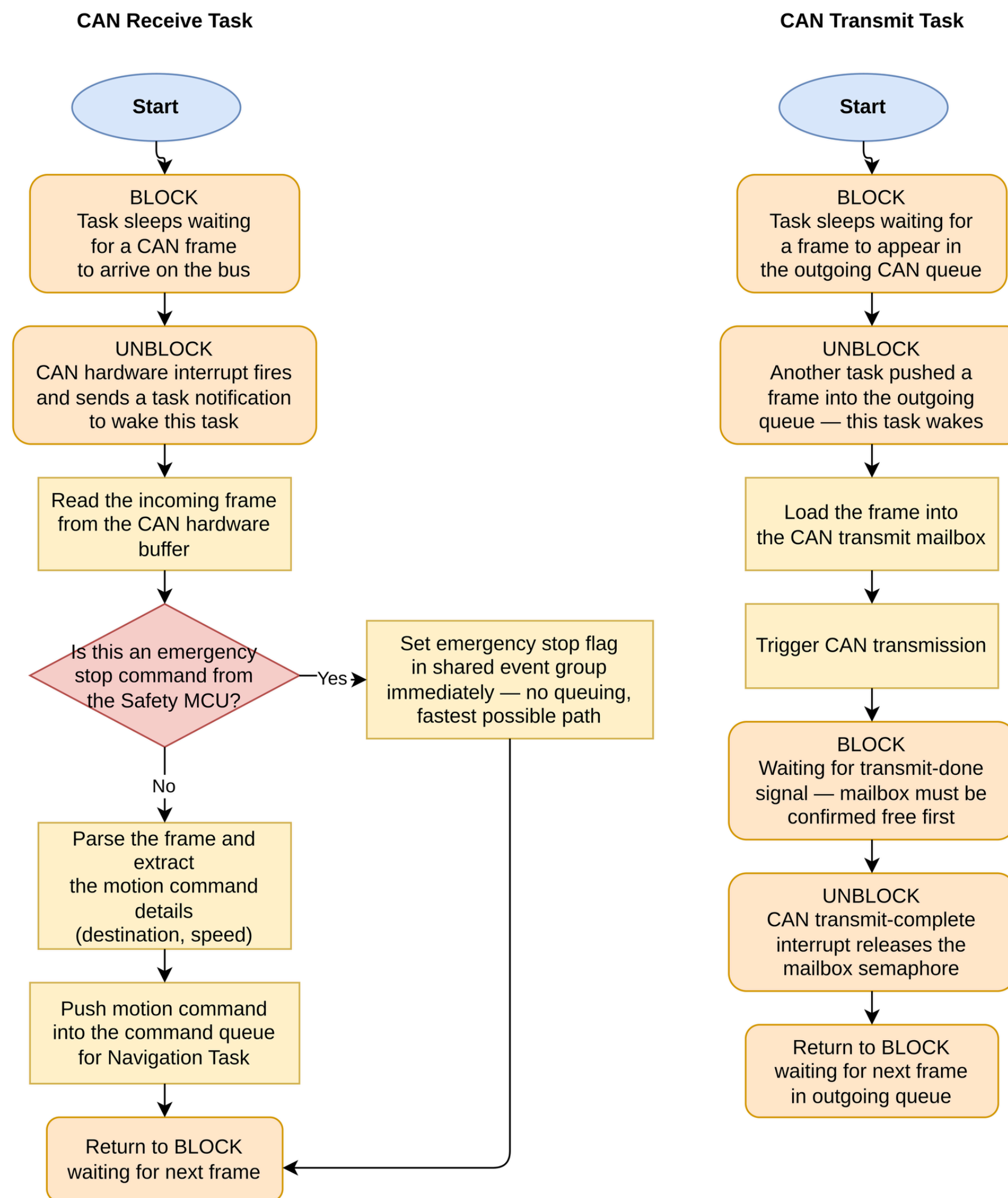
Message ID Structure (11-bit):

- Bits [10:8] = Sender Node ID
- Bits [7:0] = Message Type

Node	ID
ESP32	0x01
STM32H5	0x02
STM32H5	0x03

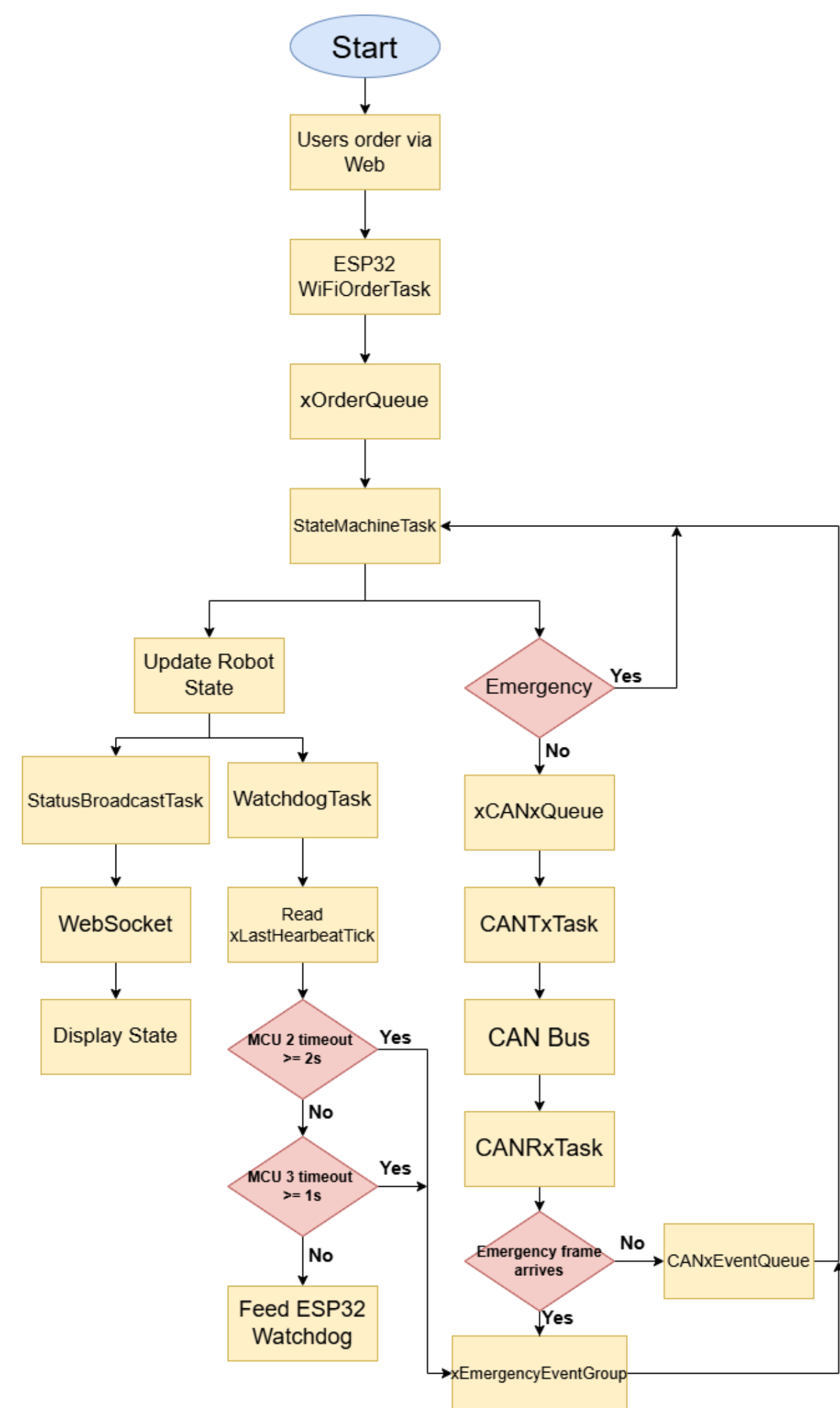
Type	MCU1 (0x1xx)	MCU2 (0x2xx)	MCU3 (0x3xx)
0x01	NAV_CMD → MCU2	NAV_STATUS → MCU1	EMERGENCY_STOP → MCU2
0x02	RETURN_BASE → MCU2	ARRIVED → MCU1	OBSTACLE_ALERT → MCU1
0x03	SOFT_STOP → MCU2	ODOMETRY → MCU1	PATH_CLEAR → MCU1
0x04	PERCEPTION_ARM → MCU3	BUMP_ALERT → MCU1	PERCEPTION_REPORT → MCU1
0x05	HEARTBEAT → ALL	HEARTBEAT → MCU1	HEARTBEAT → MCU1

2.2. Flow chart



2.2. Flow chart

ESP32 (MCU1)
— Brain, WiFi,
Mission
Controller,
OLED Display



2.2. Flow chart

State machine task

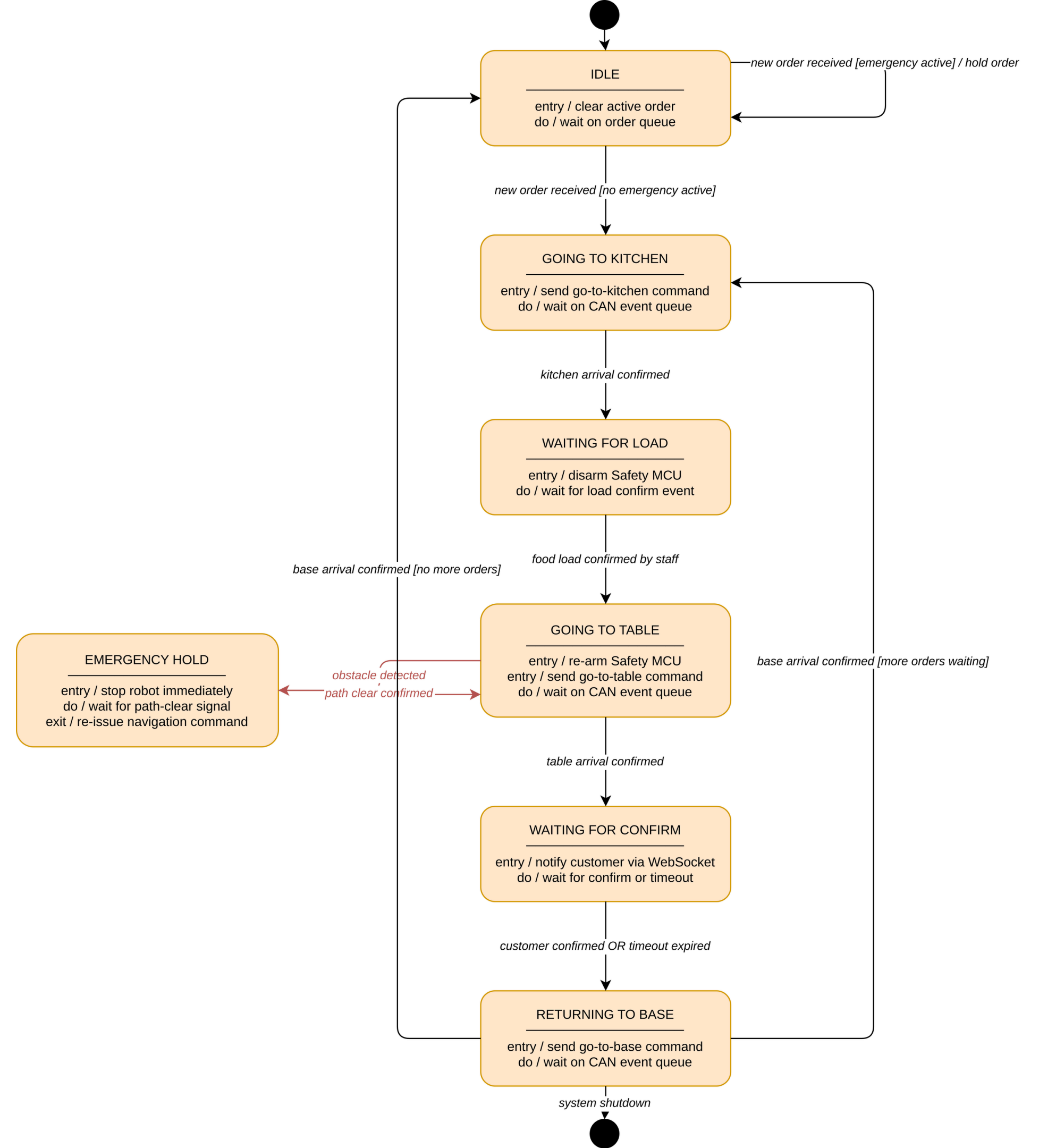
ESP32 (MCU1)

— Brain, WiFi,

Mission

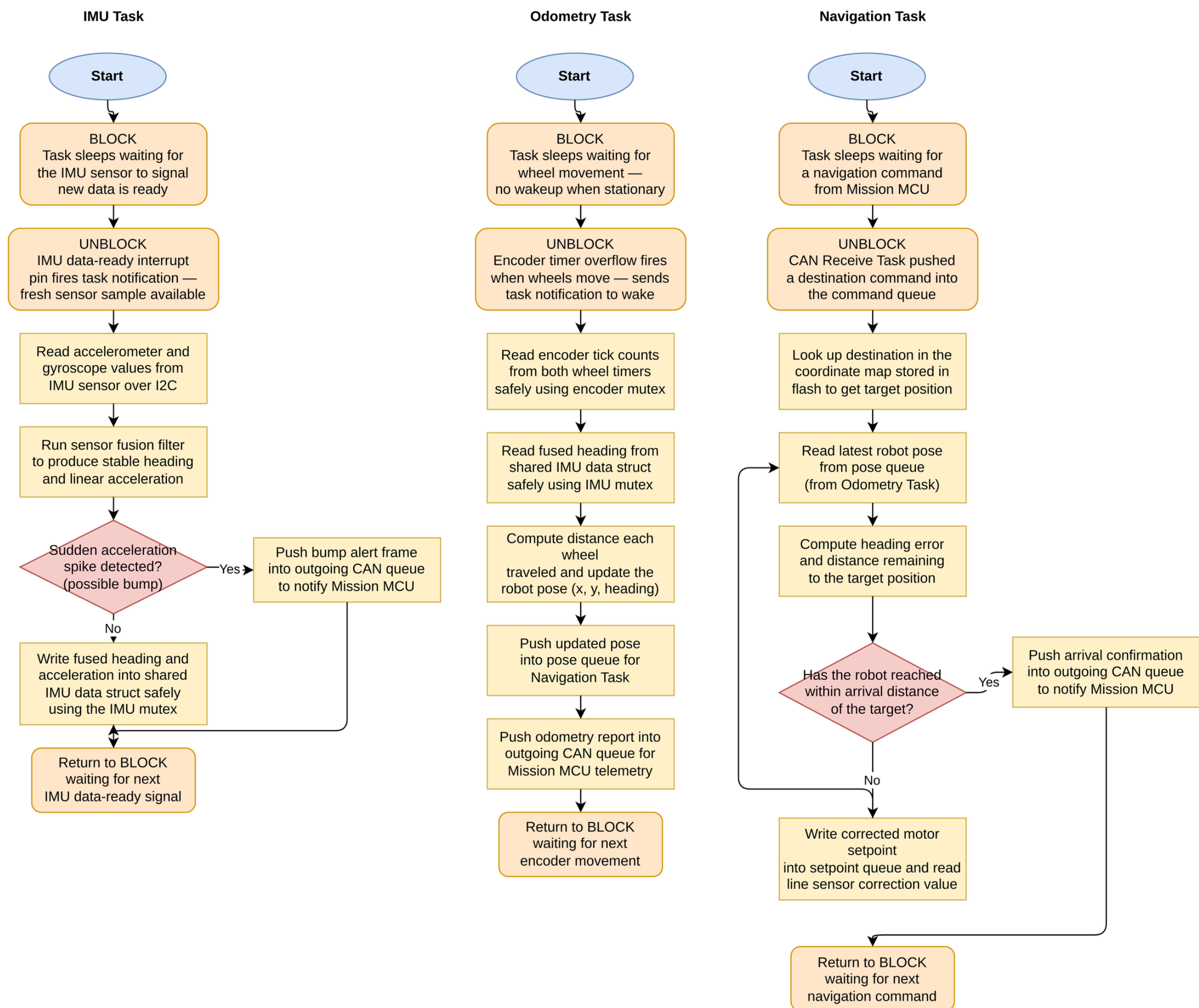
Controller,

OLED Display



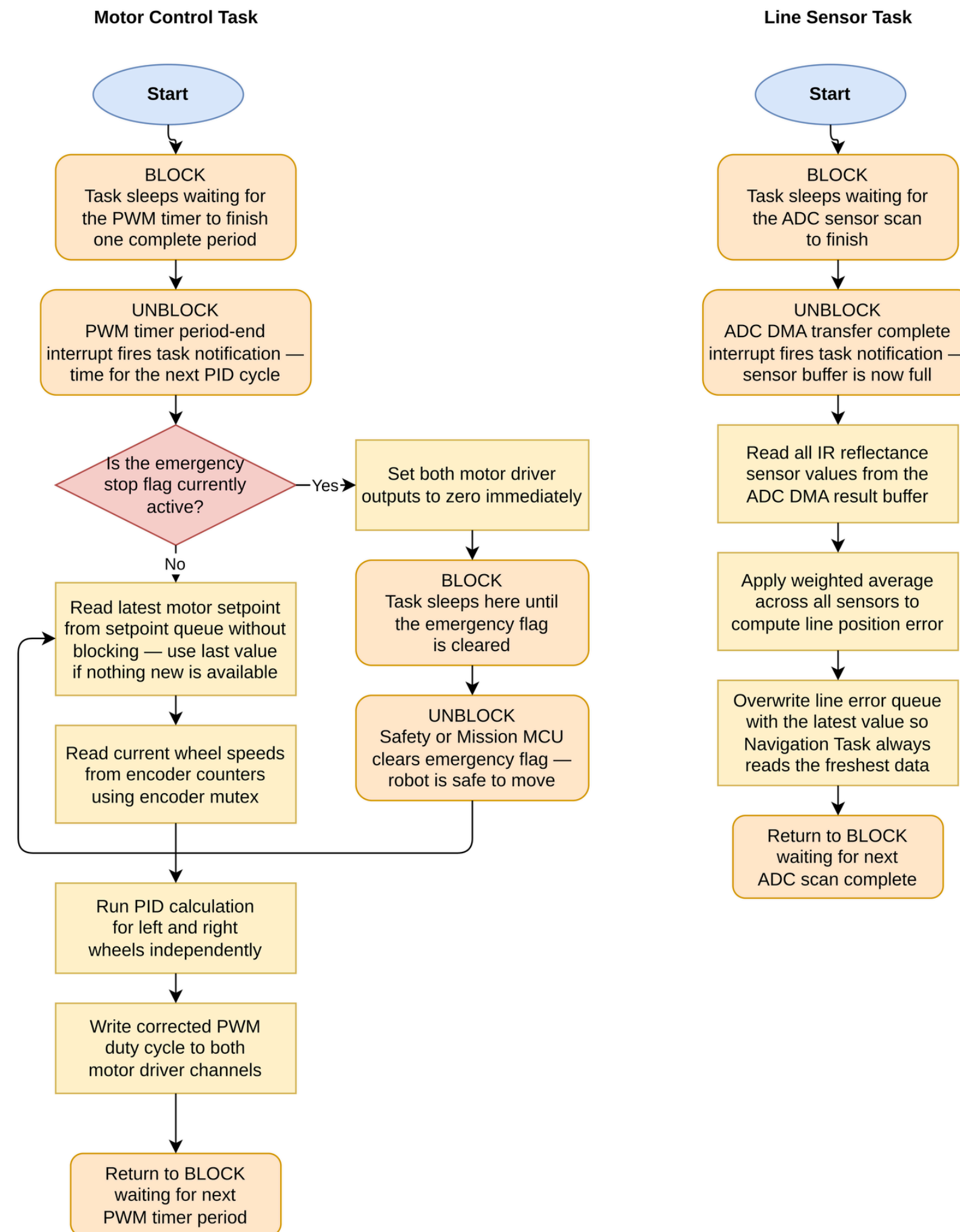
2.2. Flow chart

STM32H5 (MCU2) — Motion & Navigation Controller



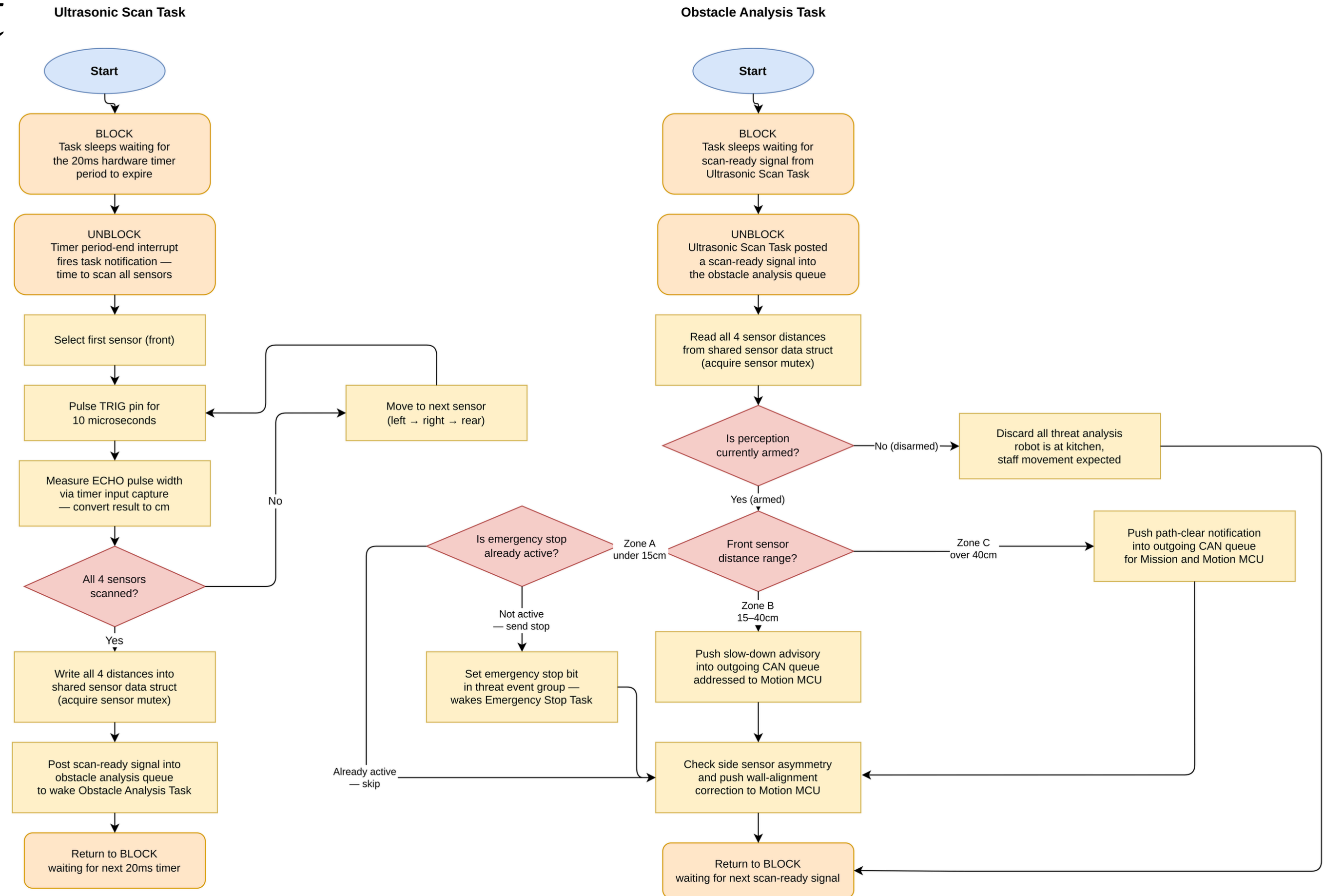
2.2. Flow chart

STM32H5 (MCU2) — Motion & Navigation Controller



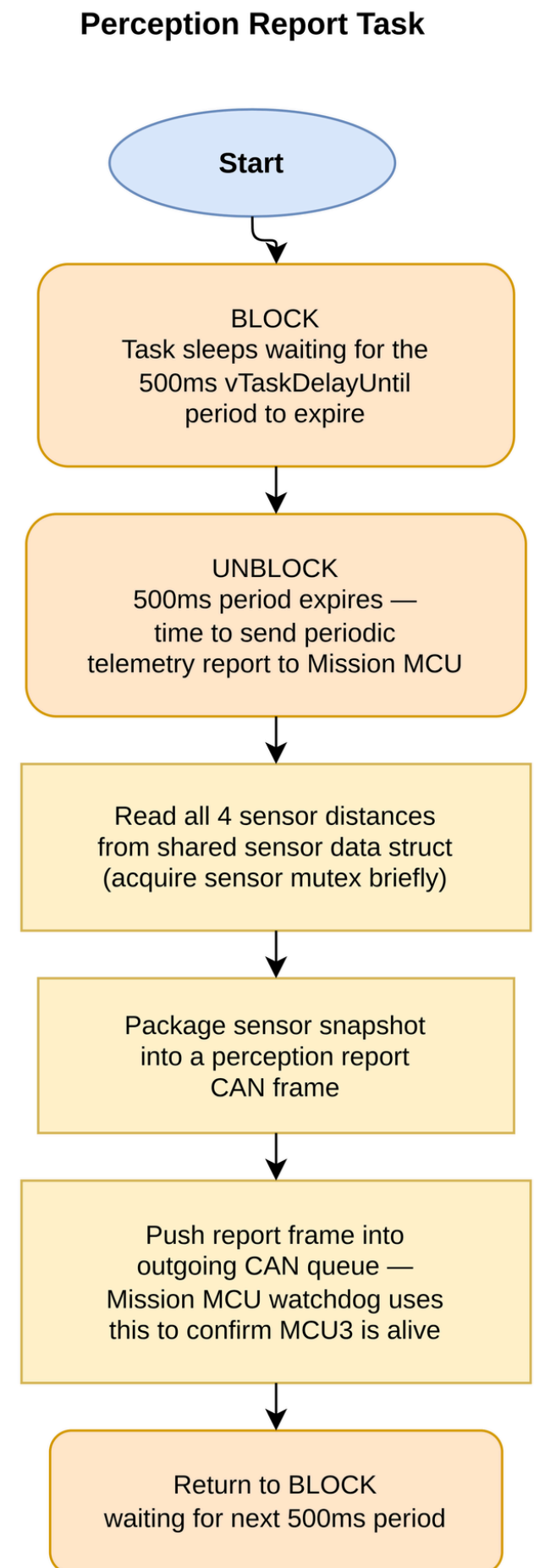
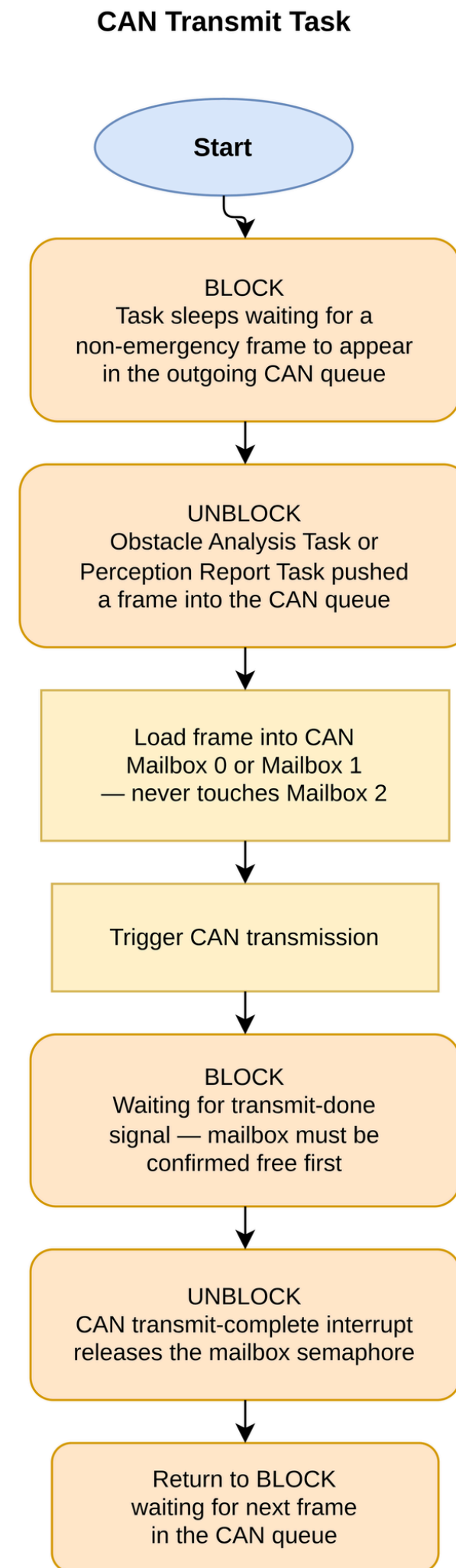
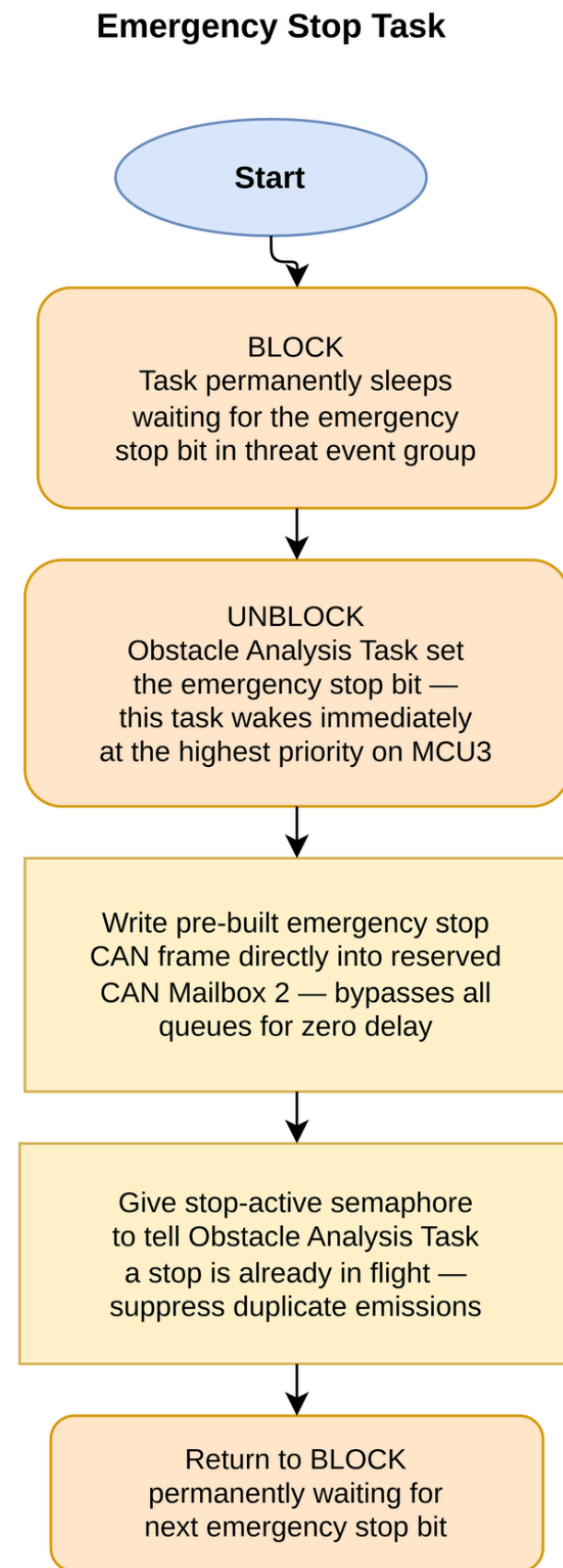
2.2. Flow chart

STM32H5 (MCU3) — Perception & Safety Controller

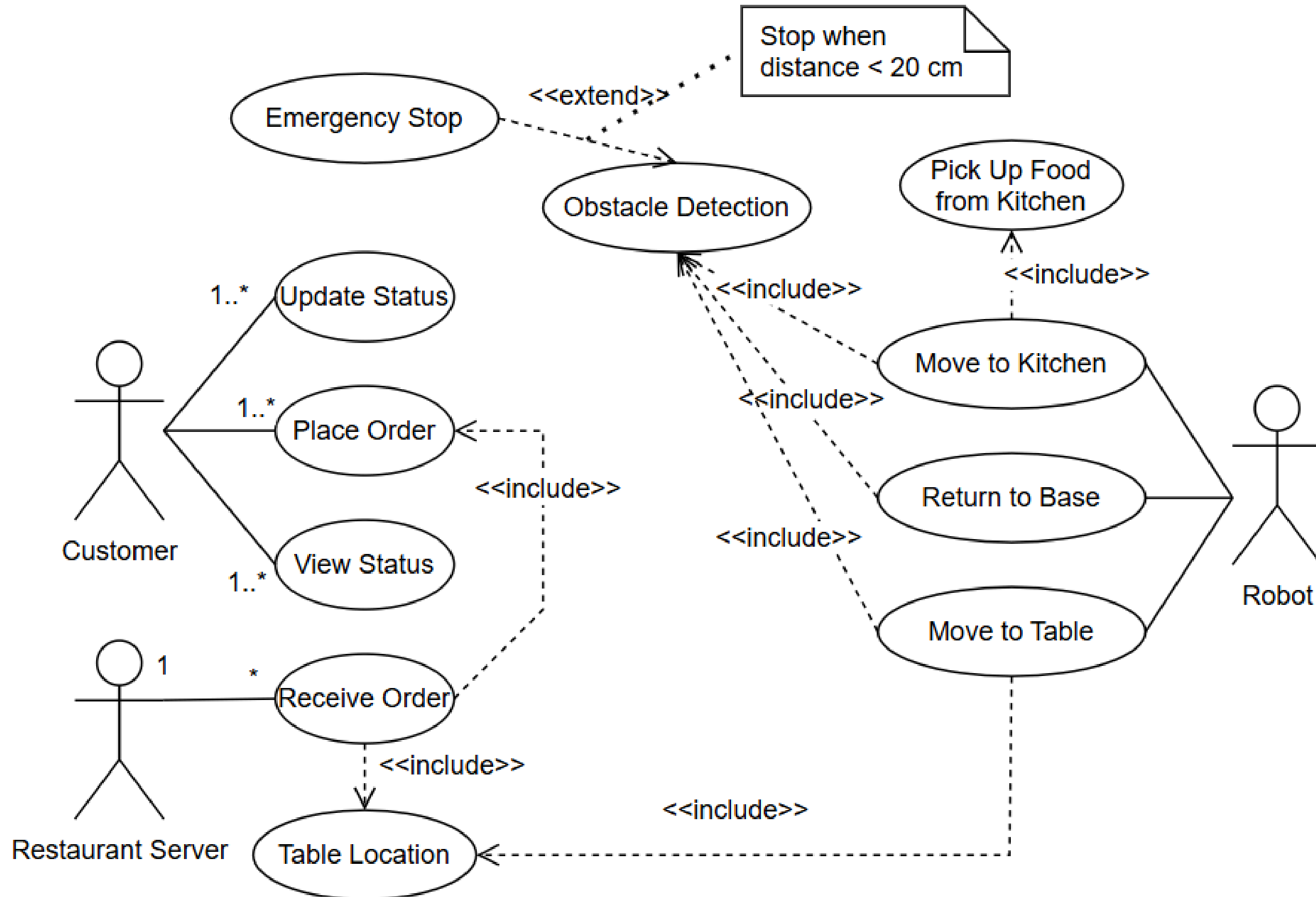


2.2. Flow chart

STM32H5 (MCU3) — Perception & Safety Controller

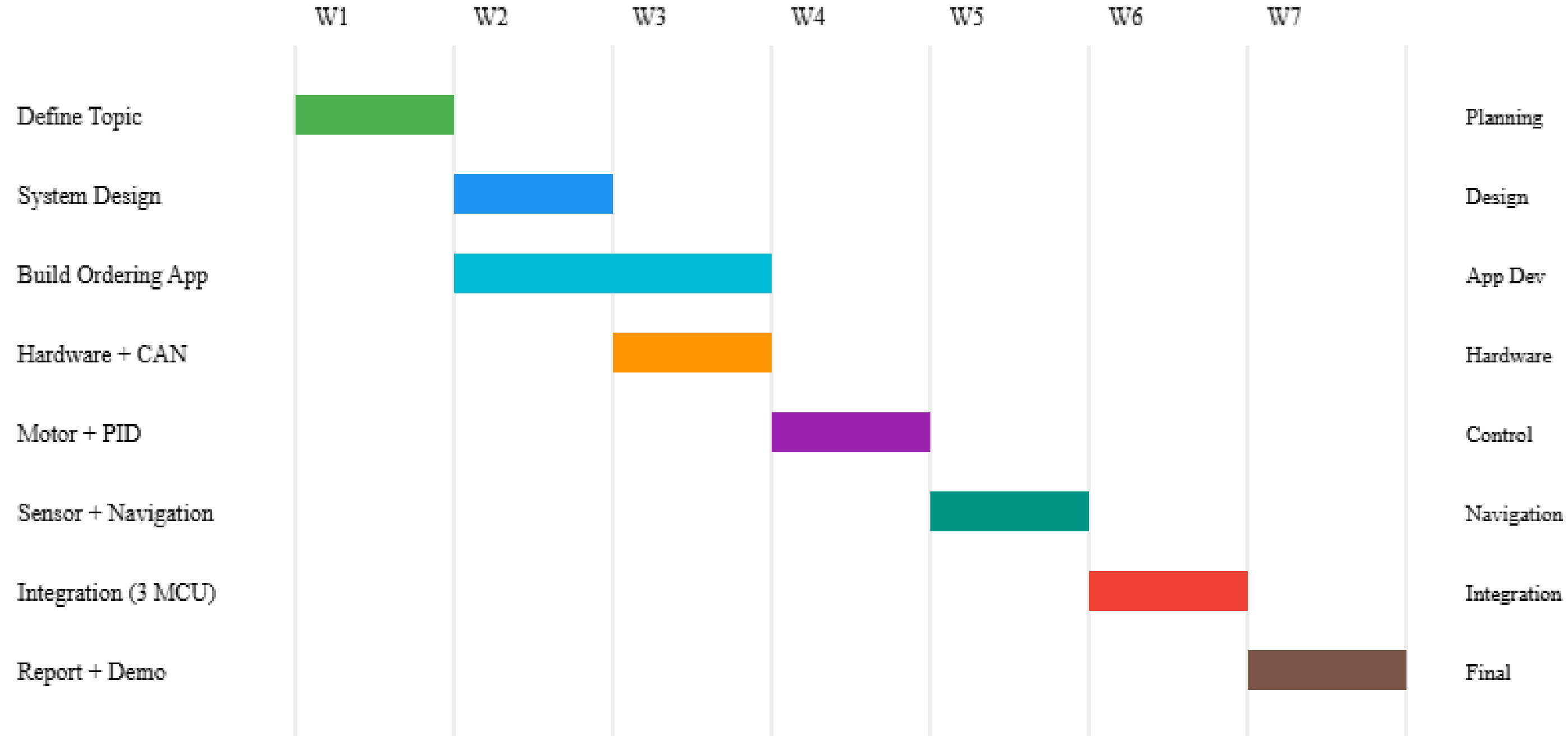


3. Use case diagram



4. Timeline

Project Timeline - Robot Delivery System (Group-3)



4. Timeline

Week 1 — Topic Definition:

- Finalize topic: Service Robot
- Write:
 - Problem Statement
 - Introduction
 - Objectives
- Design:
 - Draw system block diagram (3 MCUs)
 - List components
- Output:
 - Proposal slides
 - System diagram

Week 2 — System Design + App/Web

- Embedded DesignSystem architecture design:
 - CAN ID table (0x101, 0x102, 0x302...)
 - FreeRTOS task design for each MCU
- App / Web DevelopmentBuild a simple web interface:
 - Table selection
 - Order submission
- ESP32 side:
 - HTTP server or WebSocket server
 - Receive order → push to queue
- Expected Output:Detailed system architecture diagram
- CAN message table
- Working web application

4. Timeline

Week 3 — Hardware + CAN + Testing

- Hardware Connection:
 - ESP32 ↔ STM32 via CAN Bus
- Testing:
 - Send command from web app → ESP32 → CAN Bus
 - CAN communication verification
 - Motor running via PWM
 - Expected Output: Motor running successfully
 - CAN send/receive verified and working

Week 4 — MOTOR CONTROL + PID:

- Read encoder
- Implement PID:
 - Speed control
- Test:
 - Move straight
 - Turn left/right
- Output:
 - Stable robot movement

4. Timeline

Week 5 — SENSOR + NAVIGATION:

- Line sensor:
 - Read ADC
 - Calculate error
- Ultrasonic sensor:
 - Measure distance
- Logic:
 - Line following
 - Obstacle avoidance
- Output:
 - Autonomous robot with obstacle avoidance

Week 6 — Full 3-MCU Integration

- Add MCU3:
 - Ultrasonic sensors + emergency stop logic
- Implement:
 - Emergency stop via CAN bus
- MCU1:
 - State machine implementation
- Expected Output:
 - Robot able to:
 - Receive commands
 - Navigate and move
 - Stop when obstacle detected

4. Timeline

Week 7 — Finalization + Demo:

- Debug the entire system
- Prepare:
 - Slides
 - Report
 - Demo video
- Output:
 - Complete demo
 - Slides + report

Student	Suggested Tasks	Contribution
Bùi Trịnh Thế Viên	Write problem, introduction, and app concept, Develop web app and ESP32 WiFi communication,Test app → ESP32 → CAN command flow, Send control commands from app, Improve UI, Implement state machine, Finalize web app and demo, design PCB	25%
Trần Đăng Bách	Research sensors and safety mechanism, Design obstacle detection logic, Test ultrasonic sensors, Support sensor testing and integration, Implement obstacle avoidance, Implement emergency stop, Perform safety testing	25%
Nguyễn Văn Quý	Design system architecture and motor selection, Design PID and motor control logic, Test motor PWM and encoder reading, Implement PID and test robot movement, Develop navigation and line following, Optimize motor control, Tune robot performance	25%
Nguyễn Trọng Nhân	Prepare proposal slides and system overview, Design CAN communication and FreeRTOS tasks, Setup and debug CAN communication, Debug timing and system communication, Synchronize data between MCUs, Integrate full system (3 MCUs), Prepare report, slides, and video	25%